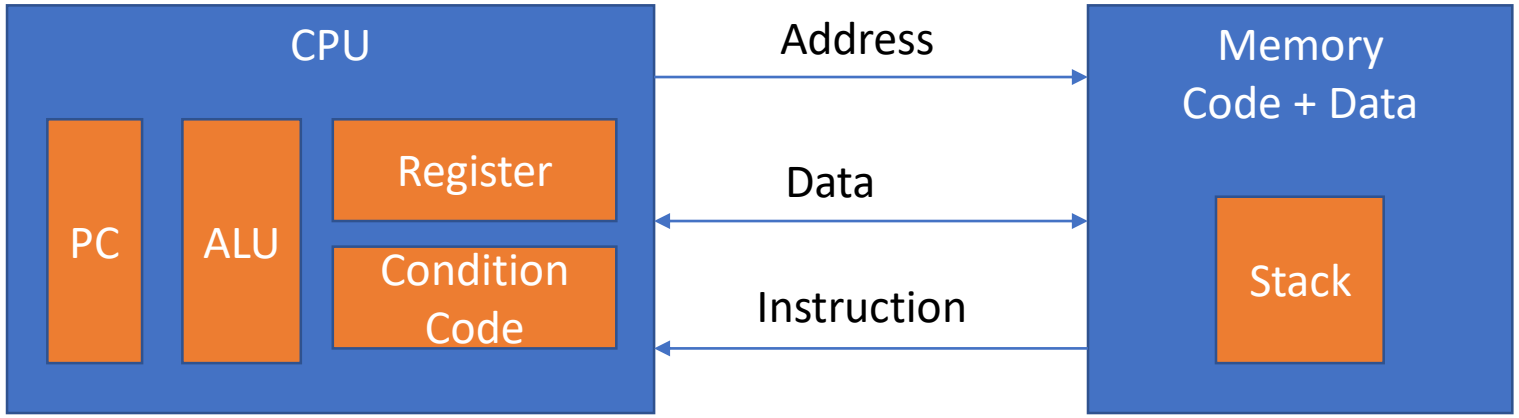
Instruction Set Architecture, Assembly Code

Machine language

- Machine language:
  - registers store collections of bits
  - all data and instructions must be encoded as collections of bits (*binary*)
  - bits are represented as electrical charges (more or less)
  - control logic and arithmetic operations are implemented as circuits, which are driven by the movement of electrical charges
  - so, the instructions directly manipulate the underlying hardware
- The collection of all valid binary instructions is known as the *machine language*.

The Abstraction Machine



PC- Program counter

PC point to next instruction

Condition codes- helps in taking decision such as condition and loop

Example of instruction set

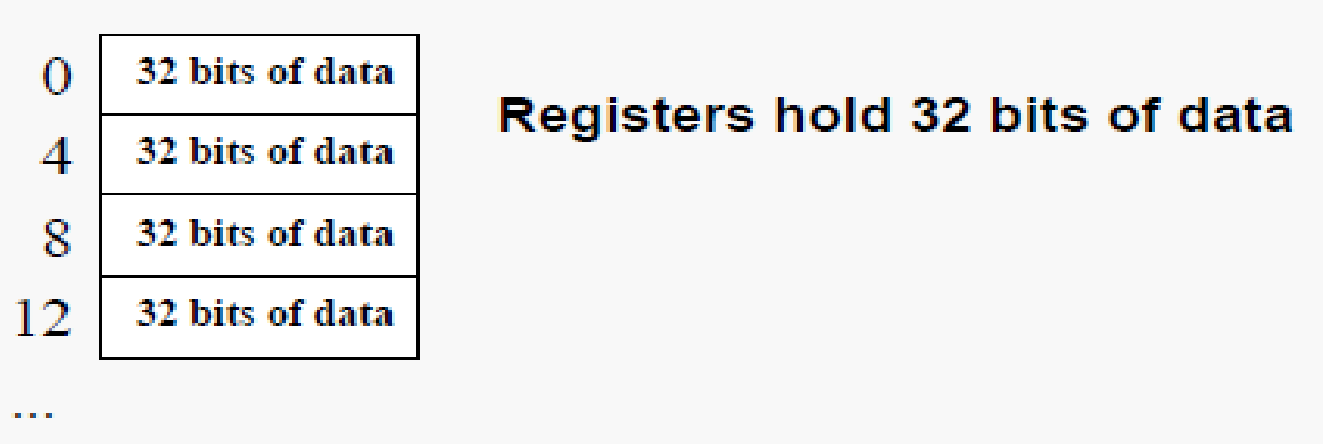
MIPS (Microprocessor without Interlocked Pipeline Stages)

Microprocessor without Interlocked Pipeline Stages (MIPS) refers to a pioneering RISC processor architecture known for its simplicity and high performance, where the "MIPS" acronym highlights its key design choice: the hardware doesn't automatically handle data dependencies (hazards), instead leaving that optimization to the compiler to insert NOPs (No-Operations) or rearrange instructions, reducing hardware complexity and enabling faster clock speeds by avoiding hardware stalls.

- Real and simple to understand
- Used by Sony play station, silicon graphics, NEC (National Electrical Code)

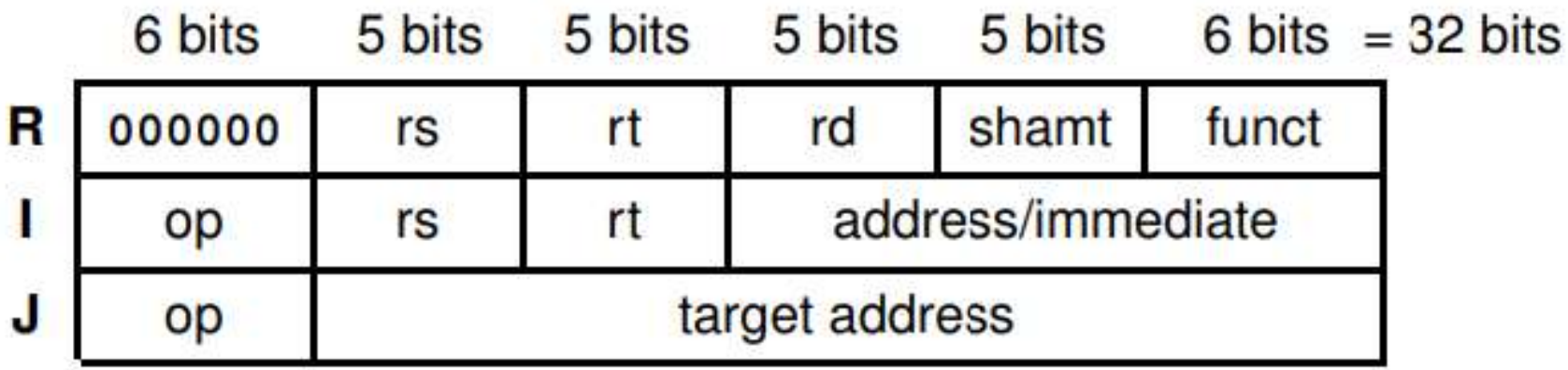
MIPS Memory Organization

- Bytes are nice, but most data items use larger "words"
- For MIPS, a *word* is 32 bits or 4 bytes.
- 2^{32} bytes with byte addresses from 0 to $2^{32} - 1$
- 2^{30} words with byte addresses 0, 4, 8, ... $2^{32} - 4$
- Words are *aligned*, that is, each has an address that is a multiple of 4.



MIPS instruction formats

Every assembly language instruction is translated into a machine code instruction in one of three **formats**



- **R**egister-type
- **I**mmediate-type
- **J**ump-type

Example instructions for each format

Register-type instructions

```
# arithmetic and logic
add   $t1, $t2, $t3
or    $t1, $t2, $t3
slt   $t1, $t2, $t3

# mult and div
mult  $t2, $t3
div   $t2, $t3

# move from/to
mfhi  $t1
mflo  $t1

# jump register
jr    $ra
```

Immediate-type instructions

```
# immediate arith and logic
addi  $t1, $t2, 345
ori   $t1, $t2, 345
slti  $t1, $t2, 345

# branch and branch-zero
beq   $t2, $t3, label
bne   $t2, $t3, label
bgtz  $t2, label

# load/store
lw    $t1, 345($t2)
sw    $t2, 345($t1)
lb    $t1, 345($t2)
sb    $t2, 345($t1)
```

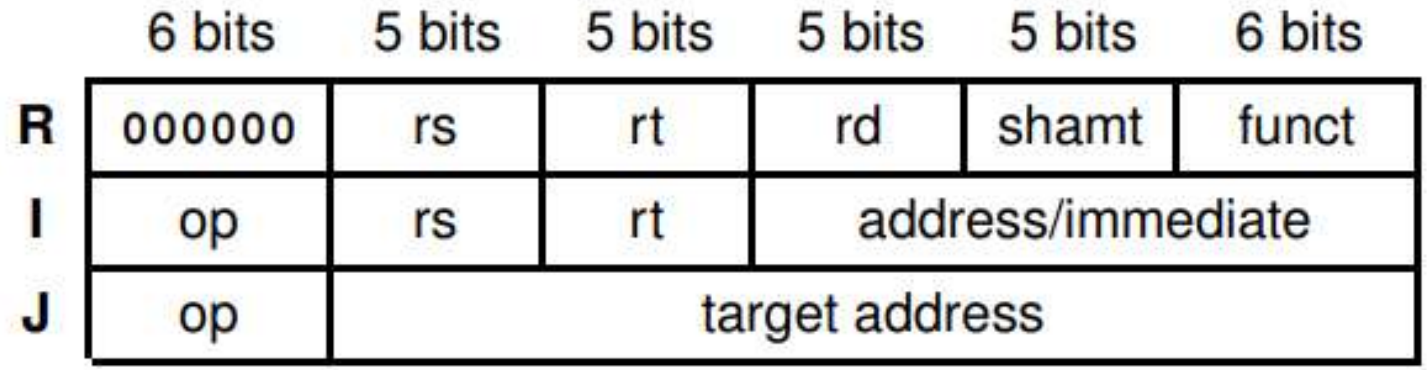
Jump-type instructions

```
# unconditional jump
j     label

# jump and link
jal   label
```

slt (Set if Less Than)
beq (Branch if Not Equal)
bne (Branch if Not Equal)
bgtz (**Branch on Greater Than Zero)**

Components of an instruction



Component	Description
op, funct	codes that determine operation to perform
rs, rt, rd	register numbers for args and destination
shamt, imm, addr	values embedded in the instruction

R-type Instructions

Components of an R-type instruction

R:	000000	rs	rt	rd	shamt	funct

R-type instruction

• op	6 bits	always zero!
• rs	5 bits	1st argument register
• rt	5 bits	2nd argument register
• rd	5 bits	destination register
• shamt	5 bits	used in shift instructions (for us, always 0s)
• funct	6 bits	code for the operation to perform
	32 bits	

Note that the destination register is third in the machine code!

Assembling an R-type instruction

```
add $t1, $t2, $t3
```

000000	rs	rt	rd	shamt	funct

rs = 10 ($t2 = $10)
rt = 11 ($t3 = $11)
rd = 9 ($t1 = $9)
funct = 32 (look up function code for add)
shamt = 0 (not a shift instruction)

000000	10	11	9	0	32
000000	01010	01011	01001	00000	100000

0000 0001 0100 1011 0100 1000 0010 0000

0x014B4820

Name	Format	Layout						Example
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
		op	rs	rt	rd	shamt	funct	
add	R	0	2	3	1	0	32	add $1, $2, $3
addu	R	0	2	3	1	0	33	addu $1, $2, $3
sub	R	0	2	3	1	0	34	sub $1, $2, $3
subu	R	0	2	3	1	0	35	subu $1, $2, $3
and	R	0	2	3	1	0	36	and $1, $2, $3
or	R	0	2	3	1	0	37	or $1, $2, $3
nor	R	0	2	3	1	0	39	nor $1, $2, $3
slt	R	0	2	3	1	0	42	slt $1, $2, $3
sltu	R	0	2	3	1	0	43	sltu $1, $2, $3

NOTE: op is 0, so funct disambiguates

`add $s0, $s1, $s2`

(registers 16, 17, 18)

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
000000	10001	10010	10000	00000	100000

NOTE: Order of components in machine code is different from assembly code. Assembly code order is similar to C, destination first. Machine code has destination last.

C: `a = b + c`

assembly code: `add $s0, $s1, $s2  # add rd, rs, rt`

machine code: `000000 10001 10010 10000 0000 100000`

`(op rs rt rd shamt funct)`

R-format Exceptions

Still have opcode 0 (all of them!), 3 registers, a shift amount, and a funct code.

Name	Format	Layout						Example
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
		op	rs	rt	rd	shamt	funct	
sll	R	0	0	2	1	10	0	sll $1, $2, 10
srl	R	0	0	2	1	10	2	srl $1, $2, 10
jr	R	0	31	0	0	0	8	jr $31

NOTE: op is 0, so funct disambiguates

`srl $s0, $s1, 1`

(registers 16, 17)

op	rs	rt	rd	shamt	funct
0	0	17	16	1	2
000000	00000	10001	10000	00001	000010

`jr $ra`

(register 31)

op	rs	rt	rd	shamt	funct
0	31	0	0	0	8
000000	11111	00000	00000	00000	001000

Exercises

Assemble the following instructions:

- `sub  $s0, $s1, $s2`
- `mult $a0, $a1`
- `jr   $ra`

Name	Number
`$zero`	0
`$v0–$v1`	2–3
`$a0–$a3`	4–7
`$t0–$t7`	8–15
`$s0–$s7`	16–23
`$t8–$t9`	24–25
`$sp`	29
`$ra`	31

Instr	fn
`add`	32
`sub`	34
`mult`	24
`div`	26
`jr`	8

I-type Instructions

Components of an I-type instruction

I-type instruction

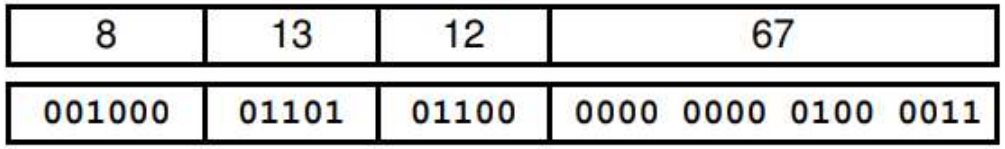
• op	6 bits	code for the operation to perform
• rs	5 bits	1st argument register
• rt	5 bits	destination or 2nd argument register
• imm	16 bits	constant value embedded in instruction
	32 bits	

Note the destination register is second in the machine code!

Assembling an I-type instruction

```
addi $t4, $t5, 67
```

op = 8 (look up op code for `addi`)
rs = 13 (`$t5` = `$13`)
rt = 12 (`$t4` = `$12`)
imm = 67 (constant value)




```
0010 0001 1010 1100 0000 0000 0100 0011

0x21AC0043
```

Instructions that follow typical format: Have 2 registers and a constant value immediately present in the instruction.

- rs: source register (5 bits)
- rt: destination register (5 bits)
- immediate value (16 bits)

Name	Format	Layout						Example
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
		op	rs	rt	immediate			
addi	I	8	2	1	100			addi $1, $2, 100
addiu	I	9	2	1	100			addiu $1, $2, 100
andi	I	12	2	1	100			andi $1, $2, 100
ori	I	13	2	1	100			ori $1, $2, 100
slti	I	10	2	1	100			slti $1, $2, 100
sltiu	I	11	2	1	100			sltiu $1, $2, 100

NOTE: *Order of components in machine code is different from assembly code. Assembly code order is similar to C, destination first. Machine code has source register, then destination.*

Exceptions: `beq`, `bne`, `lw`, `sw`, `lui`

Exceptions:

`beq, bne, lw, sw, lui`

Still have 2 registers and a constant value immediately present in the instruction.

- rs: operand or base address (5 bits)
- rt: operand or data register (5 bits)
- immediate: value or offset (16 bits)

Name	Format	Layout						Example
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
		op	rs	rt	immediate			
beq	I	4	1	2	25 (offset)			beq $1, $2, 100
bne	I	5	1	2	25 (offset)			bne $1, $2, 100
lui	I	15	0	1	100			lui $1, 100
lw	I	35	2	1	100 (offset)			lw $1, 100($2)
sw	I	43	2	1	100 (offset)			sw $1, 100($2)

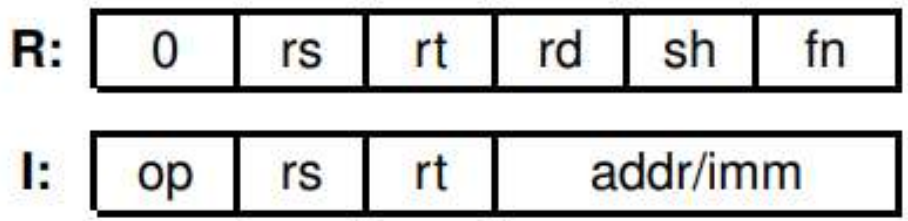
`lw $t0, 32($s3)` (registers 8 and 19)

op	rs	rt	immediate
35	19	8	32
100011	10011	01000	0000000000100000

`lui $t0, 1028` (register 8)

op	rs	rt	immediate
15	0	8	1028
001111	00000	01000	0000010000000100

Exercises



Assemble the following instructions:

- `or $s0, $t6, $t7`
- `ori $t8, $t9, 0xFF`

Name	Number
`$zero`	0
`$v0–$v1`	2–3
`$a0–$a3`	4–7
`$t0–$t7`	8–15
`$s0–$s7`	16–23
`$t8–$t9`	24–25
`$sp`	29
`$ra`	31

Instr	op/fn
`and`	36
`andi`	12
`or`	37
`ori`	13

Conditional branch instructions

```
beq $t0, $t1, label
```

I-type instruction

• op	6 bits	code for the comparison to perform
• rs	5 bits	1st argument register
• rt	5 bits	2nd argument register
• imm	16 bits	**jump offset** embedded in instruction
	32 bits	

Calculating the jump offset

Jump offset

Number of instructions from the **next instruction**

(`nop` is an instruction that does nothing)

```
        beq  $t0, $t1, skip
        nop  # 0 (start here)
        nop  # 1
        nop  # 2
skip:   nop  # 3!
        ...
```

offset = 3

```
loop:   nop  # -5
        nop  # -4
        nop  # -3
        nop  # -2
        beq  $t0, $t1, loop
        nop  # 0 (start here)
```

offset = -5

Assembling a conditional branch instruction

```
        beq  $t0, $t1, label
        nop
        nop
label:  nop
```

op	rs	rt	address/immediate

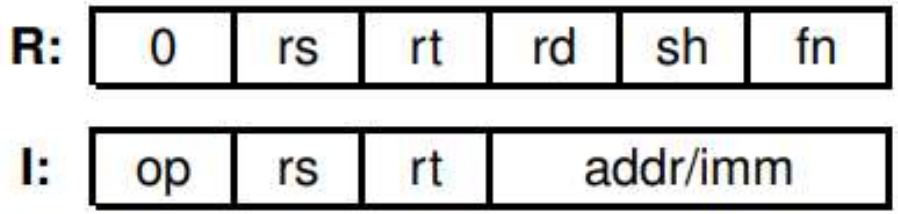
op = 4 (look up op code for `beq`)
rs = 8 (`$t0` = `$8`)
rt = 9 (`$t1` = `$9`)
imm = 2 (jump offset)

4	8	9	2
000100	01000	01001	0000 0000 0000 0010

0001 0001 0000 1001 0000 0000 0000 0010

0x11090002

Exercises



Assemble the following program:

```
# Pseudocode:
#   do {
#     i++
#   } while (i != j);
loop:  addi  $s0, $s0, 1
       bne   $s0, $s1, loop
```

Name	Number
$zero	0
$v0–$v1	2–3
$a0–$a3	4–7
$t0–$t7	8–15
$s0–$s7	16–23
$t8–$t9	24–25
$sp	29
$ra	31

Instr	op/fn
add	32
addi	8
beq	4
bne	5

J-type Instructions

J-format Instructions:

Have an address (part of one, actually) in the instruction.

```
j LOOP
```

Name	Format	Layout						Example
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
		op	address					
j	J	2	2500					j 10000
jal	J	3	2500					jal 10000

Thank you